Biostrings

Kasper D. Hansen

Dependencies

This document has the following dependencies:

```
library(Biostrings)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("Biostrings"))
```

Corrections

Improvements and corrections to this document can be submitted on its GitHub in its repository.

Overview

The *Biostrings* package contains classes and functions for representing biological strings such as DNA, RNA and amino acids. In addition the package has functionality for pattern matching (short read alignment) as well as a pairwise alignment function implementing Smith-Waterman

local alignments and Needleman-Wunsch global alignments used in classic sequence alignment (see (Durbin et al. 1998) for a description of these algorithms). There are also functions for reading and writing output such as FASTA files.

Other Resources

Representing sequences

There are two basic types of containers for representing strings. One container represents a single string (say a chromosome or a single short read) and the other container represents a set of strings (say a set of short reads). There are different classes intended to represent different types of sequences such as DNA or RNA sequences.

```
dna1 <- DNAString("ACGT-N")
dna1
```

```
##   6-letter "DNAString" instance
## seq: ACGT-N
```

```
DNAStringSet("ADE")
```

```
## Error in .Call2("new_XString_from_CHARACTER", classname, x, start(sol
```

```
dna2 <- DNAStringSet(c("ACGT", "GTCA", "GCTA"))
dna2
```

```
##   A DNAStringSet instance of length 3
##     width seq
## [1]     4 ACGT
## [2]     4 GTCA
## [3]     4 GCTA
```

Note that the alphabet of a `DNAString` is an extended alphabet: `-` (for insertion) and `N` are allowed. In fact, IUPAC codes are allowed (these codes represent different characters, for example the code “M” represents either and “A” or a “C”). A list of IUPAC codes can be

obtained by

```
IUPAC_CODE_MAP
```

```
##       A       C       G       T       M       R       W       S       Y       K
##     "A"     "C"     "G"     "T"    "AC"    "AG"    "AT"    "CG"    "CT"    "GT"
##       V       H       D       B       N
##   "ACG"   "ACT"   "AGT"   "CGT"  "ACGT"
```

Indexing into a `DNAString` retrieves a subsequence (similar to the standard R function `substr`), whereas indexing into a `DNAStringSet` gives you a subset of sequences.

```
dna1[2:4]
```

```
##   3-letter "DNAString" instance
## seq: CGT
```

```
dna2[2:3]
```

```
##   A DNAStringSet instance of length 2
##     width seq
## [1]     4 GTCA
## [2]     4 GCTA
```

Note that `[[` allows you to get a single element of a `DNAStringSet` as a `DNAString`. This is very similar to `[` and `[[` for lists.

```
dna2[[2]]
```

```
##   4-letter "DNAString" instance
## seq: GTCA
```

`DNAStringSet` objects can have names, like ordinary vectors

```
names(dna2) <- paste0("seq", 1:3)
dna2
```

```
##   A DNAStringSet instance of length 3
##     width seq                                               names
## [1]     4 ACGT                                              seq1
## [2]     4 GTCA                                              seq2
## [3]     4 GCTA                                              seq3
```

The full set of string classes are

- `DNAString[Set]`: DNA sequences.
- `RNAString[Set]`: RNA sequences.
- `AAString[Set]`: Amino Acids sequences (protein).
- `BString[Set]`: “Big” sequences, using any kind of letter.

In addition you will often see references to `XString[Set]` in the documentation. An `XString[Set]` is basically any of the above classes.

These classes seem very similar to standard `characters()` from base R, but there are important differences. The differences are mostly about efficiencies when you deal with either (a) many sequences or (b) very long strings (think whole chromosomes).

Basic functionality

Basic character functionality is supported, like

- `length`, `names`.
- `c` and `rev` (reverse the sequence).
- `width`, `nchar` (number of characters in each sequence).
- `==`, `duplicated`, `unique`.
- `as.charcater` or `toString`: converts to a base `character()` vector.
- `sort`, `order`.
- `chartr`: convert some letters into other letters.
- `subseq`, `subseq<-`, `extractAt`, `replaceAt`.
- `replaceLetterAt`.

Examples

```
width(dna2)
```

```
## [1] 4 4 4
```

```
sort(dna2)
```

```
##   A DNAStringSet instance of length 3
##     width seq                                               names
## [1]     4 ACGT                                              seq1
## [2]     4 GCTA                                              seq3
## [3]     4 GTCA                                              seq2
```

```
rev(dna2)
```

```
##   A DNAStringSet instance of length 3
##     width seq                                               names
## [1]     4 GCTA                                              seq3
## [2]     4 GTCA                                              seq2
## [3]     4 ACGT                                              seq1
```

```
rev(dna1)
```

```
##   6-letter "DNAString" instance
## seq: N-TGCA
```

Note that `rev` on a `DNAStringSet` just reverse the order of the elements, whereas `rev` on a `DNAString` actually reverse the string.

Biological functionality

There are also functions which are related to the biological interpretation of the sequences, including

- `reverse`: reverse the sequence.
- `complement`, `reverseComplement`: (reverse) complement the sequence.
- `translate`: translate the DNA or RNA sequence into amino acids.

```
translate(dna2)
```

```
##   A AAStringSet instance of length 3
##     width seq                                               names
## [1]     1 T                                                 seq1
## [2]     1 V                                                 seq2
## [3]     1 A                                                 seq3
```

```
reverseComplement(dna1)
```

```
##   6-letter "DNAString" instance
## seq: N-ACGT
```

Counting letters

We very often want to count sequences in various ways. Examples include:

- Compute the GC content of a set of sequences.
- Compute the frequencies of dinucleotides in a set of sequences.
- Compute a position weight matrix from a set of aligned sequences.

There is a rich set of functions for doing this quickly.

- `alphabetFrequency`, `letterFrequency`: Compute the frequency of all characters (`alphabetFrequency`) or only specific letters (`letterFrequency`).
- `dinucleotideFrequency`, `trinucleotideFrequency`, `oligonucleotideFrequeny`: compute frequencies of dinucleotides (2 bases), trinucleotides (3 bases) and oligonucleotides (general number of bases).
- `letterFrequencyInSlidingView`: letter frequencies, but in sliding views along the string.
- `consensusMatrix`: consensus matrix; almost a position weight matrix.

Let's look at some examples, note how the output expands to a matrix when you use the functions on a `DNAStringSet`:

```
alphabetFrequency(dna1)
```

```
## A C G T M R W S Y K V H D B N - + .
## 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0
```

```
alphabetFrequency(dna2)
```

```
##      A C G T M R W S Y K V H D B N - + .
## [1,] 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2,] 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [3,] 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
letterFrequency(dna2, "GC")
```

```
##      G|C
## [1,]   2
## [2,]   2
## [3,]   2
```

```
consensusMatrix(dna2, as.prob = TRUE)
```

```
##        [,1]      [,2]      [,3]      [,4]
## A 0.3333333 0.0000000 0.0000000 0.6666667
## C 0.0000000 0.6666667 0.3333333 0.0000000
## G 0.6666667 0.0000000 0.3333333 0.0000000
## T 0.0000000 0.3333333 0.3333333 0.3333333
## M 0.0000000 0.0000000 0.0000000 0.0000000
## R 0.0000000 0.0000000 0.0000000 0.0000000
## W 0.0000000 0.0000000 0.0000000 0.0000000
## S 0.0000000 0.0000000 0.0000000 0.0000000
## Y 0.0000000 0.0000000 0.0000000 0.0000000
## K 0.0000000 0.0000000 0.0000000 0.0000000
## V 0.0000000 0.0000000 0.0000000 0.0000000
## H 0.0000000 0.0000000 0.0000000 0.0000000
## D 0.0000000 0.0000000 0.0000000 0.0000000
## B 0.0000000 0.0000000 0.0000000 0.0000000
## N 0.0000000 0.0000000 0.0000000 0.0000000
## - 0.0000000 0.0000000 0.0000000 0.0000000
## + 0.0000000 0.0000000 0.0000000 0.0000000
## . 0.0000000 0.0000000 0.0000000 0.0000000
```

(most functions allows the return of probabilities with `as.prob = TRUE`).

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4    parallel  methods   stats     graphics  grDevices utils
## [8] datasets  base
##
## other attached packages:
## [1] Biostrings_2.36.3   XVector_0.8.0       IRanges_2.2.7
## [4] S4Vectors_0.6.3     BiocGenerics_0.14.0 BiocStyle_1.6.0
## [7] rmarkdown_0.7
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.8    formatR_1.2     magrittr_1.5    evaluate_0.7.2
##  [5] zlibbioc_1.14.0 stringi_0.5-5   tools_3.2.1     stringr_1.0.0
##  [9] yaml_2.1.13     htmltools_0.2.6 knitr_1.11
```

References

Durbin, Richard M, Sean R Eddy, Anders Krogh, Graeme Mitchison, and Sean R Eddy. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.